

DEVOPS COMMAND-LINE TOOLKIT

Every Command You Need. Every Tool You Use.
One Complete DevOps CLI Reference.



LINUX



GIT



GITHUB CLI



DOCKER



KUBECTERNES



AWS CLI



TERRAFORM



ANSIBLE



JENKINS CLI



ARGO CD CLI



PROMETHEUS



GRAFANA

```
devops@toolkit:~$  
# Automate.  
# Operate.  
# Observe.  
# Secure.  
# Ship.  
devops@toolkit:~$
```



YouTube



GitHub



LinkedIn



X



Instagram



LINUX CLI TOOLKIT

CORE OPERATING-SYSTEM COMMANDS EVERY DEVOPS ENGINEER USES



1. FILE & DIRECTORY COMMANDS

pwd	- Show current directory
ls	- List files and directories
ls -lah	- Detailed list (all, human readable)
cd <dir>	- Change directory
cd ..	- Go up one directory
cd ~	- Go to home directory
mkdir <dir>	- Create a directory
mkdir -p <a/b/c>	- Create nested directories
touch <file>	- Create an empty file
cp <src> <dest>	- Copy file
cp -r <src> <dest>	- Copy directory recursively
mv <src> <dest>	- Move/rename file or directory
rm <file>	- Remove file
rm -r <dir>	- Remove directory
rm -rf <dir>	- Force remove directory (CAUTION!)
find <path>	- Find files by name
-name <name>	
locate <name>	- Find files using database (fast)
tree	- Show directory tree structure



2. VIEW & READ COMMANDS

cat <file>	- Display file content
less <file>	- View file with paging
head <file>	- Show first 10 lines
head -n 20 <file>	- Show first 20 lines
tail <file>	- Show last 10 lines
tail -n 20 <file>	- Show last 20 lines
tail -f <file>	- Follow file in real time
wc <file>	- Count lines, words, bytes



4. PIPES & REDIRECTION

command1 command2	- Pipe output
command > file	- Redirect output (overwrite)
command >> file	- Redirect output (append)
command < file	- Input redirection
command 2> error.log	- Redirect error only
command > out.log 2>&1	- Redirect output & error
command tee file	- Pipe and save output



3. SEARCH & FILTER COMMANDS

grep "text" <file>	- Search text in file
grep -i "text" <file>	- Case-insensitive search
grep -r "text" <dir>	- Search recursively
grep -n "text" <file>	- Show line numbers
awk '{print \$1}' <file>	- Process & print fields
sed 's/old/new/g' <file>	- Stream editor (replace)
cut -d',' -f1 <file>	- Cut columns by delimiter
sort <file>	- Sort lines
uniq <file>	- Remove duplicate lines
uniq -c <file>	- Count duplicate lines
xargs <command>	- Build & execute commands
tee <file>	- Write output to file & stdout



5. PERMISSIONS & OWNERSHIP

chmod <mode> <file>	- Change permissions
Examples:	
chmod 755 <file>	- rwxr-xr-x
chmod 644 <file>	- rw-r--r--
chmod 600 <file>	- rw-----
chown <user> <file>	- Change owner
chgrp <group> <file>	- Change group
sudo <command>	- Run as superuser



6. PROCESS & SYSTEM COMMANDS

ps aux	- List all running processes
ps -ef	- Full format process list
top	- Real-time process monitor
htop	- Enhanced process viewer
kill <PID>	- Terminate process
kill -9 <PID>	- Force kill process
pkill <name>	- Kill by process name
uptime	- System uptime & load
whoami	- Current logged-in user
id	- User & group information



7. SYSTEM RESOURCES

free -h	- Memory usage (human readable)
df -h	- Disk usage
du -sh <dir>	- Directory size
du -h --max-depth=1	- Size of subdirectories
lsblk	- List block devices
vmstat 1	- Virtual memory stats
iostat -x 1	- CPU & I/O statistics
sar -u 1 3	- CPU usage (3 samples)
sar -r 1 3	- Memory usage (3 samples)



8. SERVICES & LOGS

systemctl status <svc>	- Service status
systemctl start <svc>	- Start service
systemctl stop <svc>	- Stop service
systemctl restart <svc>	- Restart service
systemctl enable <svc>	- Enable on boot
systemctl disable <svc>	- Disable on boot
journalctl	- View all logs
journalctl -u <svc>	- Logs for a service
journalctl -f	- Follow logs in real time
dmesg less	- Kernel ring buffer logs



9. REAL DEVOPS EXAMPLES USING COMMAND PIPELINES

- Find large files (> 100MB) in current directory
`find . -type f -size +100M -exec ls -lh {} \; | sort -k5 -h`
- Top 10 CPU consuming processes
`ps aux --sort=-%cpu | head -n 11`
- Find error lines in application log
`grep -i "error" app.log | tail -n 50`
- Count log lines by status code
`grep -oE "(200|404|500)" access.log | awk '{print $1}' | sort | uniq -c | sort -nr`
- Check disk usage of large directories
`du -h --max-depth=1 /var | sort -hr | head -n 10`
- Monitor a log file and filter important entries
`tail -f /var/log/syslog | grep -i "failed\[error\|denied"`



SECURITY REMINDER

Use least privilege.
Be careful with sudo.
Protect credentials & sensitive data.



CAUTION

Commands like `rm -rf`, `chmod 777`, and `kill -9` can cause data loss or service outages. Use carefully.



PRACTICE THIS

Practice in a lab environment.
Build small scripts.
Make the CLI your superpower.



DEVOPS TIP

Know the tools.
Combine them with pipes.
Automate repetitive tasks.
Always test safely.
Never use destructive commands without understanding them.

NETWORKING CLI TOOLKIT

COMMANDS FOR INVESTIGATING CONNECTIVITY FROM HOST TO APPLICATION



1. INTERFACE & IP INFORMATION

`ip` - Show all IP info
`ip addr` - Show IP addresses
`ip route` - Show routing table
`ip link` - Show network links

EXAMPLE

```
$ ip addr
$ ip route
```

Use to check IP, interface state, and routing on the host.



2. SOCKET & CONNECTIONS

`ss` - Show sockets
`ss -tulpn` - Listening ports with process

EXAMPLE

```
$ ss -tulpn
$ ss -ant
```

Use to inspect listening ports and active connections.



3. CONNECTIVITY TESTING

`ping <host>` - Test reachability
`traceroute <host>` - Trace route
`tracepath <host>` - Trace path (Linux)

EXAMPLE

```
$ ping 8.8.8.8
$ traceroute google.com
```

Use to test reachability and path to destination.



4. DNS INVESTIGATION

`dig <domain>` - DNS lookup
`dig +short <domain>` - Short answer
`nslookup <domain>` - DNS lookup
`host <domain>` - DNS lookup

EXAMPLE

```
$ dig example.com
$ dig +short example.com
```

Use to troubleshoot DNS resolution issues.



5. HTTP / WEB DEBUGGING

`curl <url>` - Send HTTP request
`curl -I <url>` - Show headers
`curl -v <url>` - Verbose output
`wget <url>` - Download file

EXAMPLE

```
$ curl -I https://example.com
$ curl -v https://example.com
```

Use to test web endpoints, headers, and responses.



6. PORT TESTING

`nc <host> <port>` - Test TCP port
`nc -zv <host> <port>` - Verbose scan
`nc -vz <host> <port>` - Scan without sending data

EXAMPLE

```
$ nc -zv example.com 443
$ nc -vz 10.0.0.10 22
```

Use to verify if a port is open and reachable.



7. PACKET CAPTURE

`tcpdump -i any` - Capture packets
`tcpdump host <ip>` - Capture to/from IP
`tcpdump port <port>` - Capture by port

EXAMPLE

```
$ sudo tcpdump -i any host 8.8.8.8
$ sudo tcpdump -i any port 80
```

Use to capture and analyze network packets.



8. TLS / SSL DEBUGGING

`openssl s_client -connect <host>:<port>` - Test TLS connection
`openssl s_client -connect <host>:<port> -servername <host>` - SNI / certificate check

EXAMPLE

```
$ openssl s_client -connect example.com:443
```

Use to inspect TLS handshake, certificates, and encryption.



9. ROUTING INVESTIGATION

`ip route` - Show routing table
`ip route get <ip>` - Show route used to reach IP

EXAMPLE

```
$ ip route
$ ip route get 8.8.8.8
```

Use to verify the route and next hop to destination.



10. SOCKET & PROCESS INSPECTION

`ss -p` - Show process info
`ss -tunap` - TCP/UDP + process
`lsof -i :<port>` - Process using port

EXAMPLE

```
$ ss -tunap | grep :80
$ lsof -i :443
```

Use to find which process is using a network port.



NETWORK TROUBLESHOOTING WORKFLOW

- 1 IDENTIFY THE PROBLEM What is not working? (e.g., slow, timeout, unreachable)
- 2 CHECK DNS `dig / nslookup / host` to verify name resolution
- 3 CHECK ROUTE `ip route / traceroute / tracepath` to verify path
- 4 CHECK FIREWALL Ensure traffic is not blocked by firewall/security groups
- 5 CHECK PORT `nc / ss` to verify the target port is open and listening
- 6 CHECK SERVICE Verify the service is running and healthy
- 7 CHECK APPLICATION `curl / wget` to test the application endpoint
- 8 CAPTURE PACKETS (IF NEEDED) Use `tcpdump` for deep packet-level analysis

DEVOPS CONNECTIVITY FLOW



DNS → ROUTE → FIREWALL → PORT → SERVICE → APPLICATION

FOLLOW THIS FLOW TO FIND WHERE THE PROBLEM EXISTS

COMMAND QUICK REFERENCE

CATEGORY	COMMAND	PURPOSE	EXAMPLE
DNS	<code>dig example.com</code>	DNS lookup	<code>dig example.com</code>
ROUTE	<code>ip route</code>	View routing table	<code>ip route</code>
CONNECTIVITY	<code>ping 8.8.8.8</code>	Test reachability	<code>ping 8.8.8.8</code>
TRACEROUTE	<code>traceroute google.com</code>	Trace path to host	<code>traceroute google.com</code>
PORT	<code>nc -zv host 443</code>	Check if port is open	<code>nc -zv example.com 443</code>
SOCKETS	<code>ss -tulpn</code>	List listening ports	<code>ss -tulpn</code>
HTTP	<code>curl -I https://example.com</code>	Check HTTP headers	<code>curl -I https://example.com</code>
TLS	<code>openssl s_client -connect host:443</code>	Test TLS connection	<code>openssl s_client -connect example.com:443</code>
PACKETS	<code>tcpdump -i any host 1.1.1.1</code>	Capture packets	<code>sudo tcpdump -i any host 1.1.1.1</code>



PRO TIP

- Start from DNS and move forward in order.
- Always test from the server/host where the problem occurs.
- Use `-v` for verbose output when debugging.
- Logs + metrics + CLI = faster root cause.



COMMON USE CASES

- Application not reachable
- Intermittent connectivity
- Service down or slow
- Port not responding
- TLS handshake failure
- DNS resolution issues

GIT CLI TOOLKIT

GIT OPERATIONS USED IN REAL DEVELOPMENT AND DELIVERY WORKFLOWS



CORE GIT COMMANDS

COMMAND	WHAT IT DOES	EXAMPLE
<code>git init</code>	Initialize a new Git repository	<code>git init</code>
<code>git clone <url></code>	Clone a remote repository	<code>git clone https://github.com/user/repo.git</code>
<code>git status</code>	Show working tree status	<code>git status</code>
<code>git add <file></code>	Stage changes	<code>git add app.py</code>
<code>git commit -m "msg"</code>	Commit staged changes	<code>git commit -m "Add login feature"</code>
<code>git diff</code>	Show changes (unstaged)	<code>git diff</code>
<code>git log</code>	Show commit history	<code>git log --oneline --graph</code>
<code>git show <commit></code>	Show details of a commit	<code>git show a1b2c3d</code>
<code>git branch</code>	List, create, or delete branches	<code>git branch -av</code>
<code>git switch <branch></code>	Switch to a branch	<code>git switch feature/login</code>
<code>git fetch</code>	Fetch changes from remote	<code>git fetch origin</code>
<code>git pull</code>	Fetch and merge changes	<code>git pull origin main</code>
<code>git push</code>	Push changes to remote	<code>git push origin main</code>
<code>git merge <branch></code>	Merge branch into current branch	<code>git merge feature/login</code>
<code>git rebase <branch></code>	Reapply commits on top of another branch	<code>git rebase main</code>
<code>git stash</code>	Stash local changes	<code>git stash -u</code>
<code>git stash pop</code>	Apply and drop the last stash	<code>git stash pop</code>
<code>git tag <name></code>	Create a tag (lightweight by default)	<code>git tag v1.0.0</code>
<code>git remote -v</code>	Show remote repositories	<code>git remote -v</code>
<code>git cherry-pick <commit></code>	Apply a specific commit	<code>git cherry-pick a1b2c3d</code>
<code>git revert <commit></code>	Create a new commit that undoes changes	<code>git revert a1b2c3d</code>
<code>git reset --soft <commit></code>	Move HEAD, keep changes staged	<code>git reset --soft HEAD~1</code>
<code>git reset --mixed <commit></code>	Move HEAD, unstage changes	<code>git reset HEAD~1</code>
<code>git reset --hard <commit></code>	Move HEAD, discard changes	<code>git reset --hard HEAD~1</code>
<code>git reflog</code>	Show reference log (recover lost commits)	<code>git reflog</code>

MERGE CONFLICT INVESTIGATION

- 1 Run git status
- See conflicted files.
- 2 Open the files
- Look for conflict markers:

```
<<<<<< HEAD
=====
>>>>>> branch-name
```
- 3 Resolve the conflict
- Edit the file, keep the correct changes.
- 4 Stage and commit
- `git add <file>`
- `git commit -m "Resolve conflict"`

COMMIT RECOVERY

- Lost commit? Use git reflog
- Find the commit hash.
- Recover with:

```
git reset --hard <hash>
```

SAFE ROLLBACK OPTIONS

Revert (Safe)

Creates a new commit that undoes changes.

```
git revert <commit>
```

Reset (Move Back)

Moves branch pointer.

```
git reset --soft|--mixed <commit>
```

Reset Hard (Discard)

Deletes changes permanently.

```
git reset --hard <commit>
```

BRANCH COMPARISON

Compare branches:

```
git diff main..feature/login
```

See what's ahead / behind:

```
git log --oneline --graph main..feature/login
```

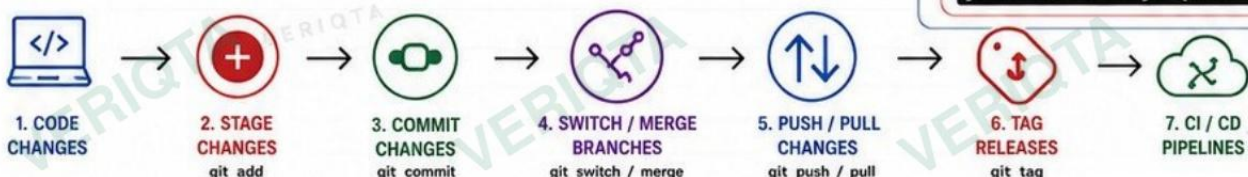
Show stats:

```
git diff --stat main..feature/login
```

RELEASE TAGGING

- 1 Create tag
`git tag -a v1.2.0 -m "Release v1.2.0"`
- 2 Push tag
`git push origin v1.2.0`
- 3 Push all tags
`git push origin --tags`
- 4 List tags
`git tag -l`

REAL DEVOPS WORKFLOW WITH GIT



REMOTE MANAGEMENT

Add remote

```
git remote add origin <url>
```

View remotes

```
git remote -v
```

Remove remote

```
git remote remove origin
```

Rename remote

```
git remote rename origin upstream
```


GITHUB CLI TOOLKIT

MANAGE GITHUB WITHOUT LEAVING THE TERMINAL



WHAT IS GITHUB CLI (gh)?

GitHub CLI is the official command-line tool to work with GitHub from your terminal.

KEY BENEFITS

- ✓ Manage repositories, PRs, Issues, Workflows, and more
- ✓ Automate DevOps workflows and CI/CD tasks
- ✓ Inspect and debug Actions runs directly
- ✓ Secure authentication with GitHub Tokens and SSH
- ✓ Powerful API access from the terminal

CORE gh COMMAND GROUPS

COMMAND GROUP	PURPOSE	EXAMPLE
gh auth	Authenticate with GitHub	<code>\$ gh auth login</code>
gh repo	Manage repositories	<code>\$ gh repo list</code>
gh pr	Work with pull requests	<code>\$ gh pr list</code>
gh issue	Manage issues	<code>\$ gh issue list</code>
gh workflow	Manage workflows	<code>\$ gh workflow list</code>
gh run	Work with workflow runs	<code>\$ gh run list</code>
gh release	Manage releases	<code>\$ gh release list</code>
gh secret	Manage repository secrets	<code>\$ gh secret list</code>
gh variable	Manage repository variables	<code>\$ gh variable list</code>
gh api	Make authenticated API calls	<code>\$ gh api repos/owner/repo</code>

AUTHENTICATION

GitHub CLI supports multiple authentication methods.

1 HTTPS AUTH (DEFAULT)

```
$ gh auth login
```

2 SSH AUTH

```
$ gh auth login --hostname github.com  
--git-protocol ssh
```

3 CHECK AUTH STATUS

```
$ gh auth status
```

Supports: GitHub.com, GitHub Enterprise, Multiple Accounts

DEVOPS AUTOMATION WITH gh

Use gh in scripts and pipelines.

- Automate PR creation and review
- Trigger and monitor CI/CD workflows
- Fetch workflow logs for debugging
- Manage releases and tags
- Handle issues and project management
- Securely access GitHub APIs
- Integrate with Bash, Python, CI/CD tools

Example in a script:

```
gh pr list --json number,title,state | \njq -r '.[] | "{number}: {title} - {state}"'
```

REAL DEVOPS EXAMPLES

- ✓ Automate PR Review: `gh pr list --state open --json number,title,author`
- ✓ CI/CD Monitoring: `gh run list --limit 5 --json databaseId,status,conclusion`
- ✓ Release Automation: `gh release create v1.2.0 --generate-notes`
- ✓ Secret Rotation Check: `gh secret list --repo owner/repo`
- ✓ Workflow Dispatch: `gh workflow run deploy.yml -f environment=staging`
- ✓ API Data Extraction: `gh api repos/owner/repo/deployments --paginate`
- ✓ DevOps Notification: `gh run watch 66 gh run view <id> --log`
- ✓ Organization Repo Audit: `gh repo list org --limit 100 --json name,visibility`

COMMON TASKS WITH gh

1 CLONE A REPOSITORY

```
$ gh repo clone owner/repo
```

2 CREATE A PULL REQUEST

```
$ gh pr create --title "Add login feature" --body "Closes #123"
```

3 VIEW PR STATUS

```
$ gh pr view 123 --json statusCheckRollup
```

4 MERGE A PULL REQUEST

```
$ gh pr merge 123 --squash --delete-branch
```

5 TRIGGER A WORKFLOW

```
$ gh workflow run build.yml -f environment=prod
```

6 WATCH WORKFLOW RUN LIVE

```
$ gh run watch
```

7 VIEW RUN DETAILS

```
$ gh run view 456789 --log
```

8 LIST FAILED JOBS

```
$ gh run view 456789 --json jobs | jq '.jobs[] | select(.conclusion=="failure")'
```

9 CREATE A RELEASE

```
$ gh release create v1.2.0 --title "v1.2.0 Release" --notes  
"Release notes here"
```

10 QUERY GITHUB API

```
$ gh api repos/owner/repo/issues --paginate
```

QUICK REFERENCE

Help for any command	<code>gh <command> --help</code>
List all commands	<code>gh</code>
View current user	<code>gh api user</code>
List your repositories	<code>gh repo list</code>
Clone repository	<code>gh repo clone owner/repo</code>
List open PRs	<code>gh pr list --state open</code>
View PR details	<code>gh pr view <number></code>
Merge PR	<code>gh pr merge <number></code>
List workflows	<code>gh workflow list</code>
List workflow runs	<code>gh run list</code>
View run logs	<code>gh run view <id> --log</code>
Watch run live	<code>gh run watch</code>
List releases	<code>gh release list</code>
Create release	<code>gh release create <tag></code>
List secrets	<code>gh secret list</code>
Make API call	<code>gh api <endpoint></code>

BASH AND SHELL AUTOMATION TOOLKIT

TURN INDIVIDUAL COMMANDS INTO
REPEATABLE DEVOPS AUTOMATION



1. VARIABLES

User-defined variables

```
name="DevOps"
count=10
path="/opt/tools"
echo $name
```

No spaces around =
Use quotes for strings



2. ENVIRONMENT VARIABLES

System or user env vars

```
export PATH=$PATH:/usr/local/bin
echo $PATH
printenv | grep HOME
```

export makes it available
to child processes



3. POSITIONAL PARAMETERS

Access script arguments

```
#!/bin/bash
echo "First: $1"
echo "Second: $2"
echo "All: $@"
echo "Count: $#"
```

\$0 is the script name



4. COMMAND SUBSTITUTION

Capture command output

```
current_user=$(whoami)
date_now=$(date +%F)
disk_free=$(df -h |
awk 'NR==2{print $4}')
echo "User: $current_user"
```

Use \$(command) preferred
over backticks



13. SHELL SAFETY OPTIONS

Write safer, more reliable scripts

```
set -e      Exit on any error
set -u      Error on undefined variables
set -o pipefail  Fail if any command in
                 a pipeline fails
set -euxo pipefail  Common strict mode
                 for robust scripts
```



14. TRAP

Catch signals and exit cleanly

```
trap 'echo "Script interrupted";
exit 1' INT TERM EXIT
```

Clean up temporary files,
release locks, notify, etc.



15. SCRIPT ARGUMENTS

Handle script inputs

```
#!/bin/bash
echo "Script: $0"
echo "Arg1: $1"
echo "Arg2: $2"
echo "All args: $@"
echo "Count: $#"
```

Use quotes with \$@ and \$*



16. ERROR HANDLING

Handle expected failures

```
if ! command; then
echo "Command failed"
exit 1
fi
```

Fail fast, give clear messages,
exit with proper codes



17. LOGGING

Record what your scripts do

```
LOGFILE="/var/log/myscript.log"
log() {
echo "[$(date +%F %T)] $1"
tee -a "$LOGFILE"
}
log "Script started"
```

Timestamp logs, keep audit trail



20. DEVOPS AUTOMATION WITH BASH

- ✓ Glue between tools
- ✓ CI/CD pipeline scripts
- ✓ Infra provisioning helpers
- ✓ Monitoring & alerting scripts
- ✓ Backup & recovery scripts
- ✓ Deployment automation

MAINTAIN

5. EXIT CODES

Check command success

```
ls /not/exist
echo $?
if [ $? -eq 0 ]; then
echo "Success"
else
echo "Failed"
fi
```

0 = success
non-zero = failure



6. CONDITIONALS

Make decisions

```
if [ $count -gt 10 ]; then
echo "Greater than 10"
elif [ $count -eq 10 ]; then
echo "Equal 10"
else
echo "Less than 10"
fi
```

CASE example

```
case $env in
dev) echo "Development";;
prod) echo "Production";;
*) echo "Unknown";;
esac
```



7. LOOPS

Repeat actions

FOR loop

```
for i in {1..5}; do
echo "Iteration $i"
done
```

WHILE loop

```
while [ $count -gt 0 ]; do
echo $count
count=$((count-1))
done
```



8. FUNCTIONS

Reusable code blocks

```
say_hello() {
echo "Hello $1"
}

say_hello "DevOps"
```

Pass arguments to
functions like scripts

9. ARRAYS

Store multiple values

```
servers=(web1 web2 web3)
echo "First: ${servers[0]}"
echo "All: ${servers[@]}"
for s in "${servers[@]"; do
ping -c 1 $s
done
```



10. PIPES

Pass output between
commands

```
ps aux | grep nginx
df -h | sort -k5 -h
cat .access.log | grep
"404" | wc -l
```

Build powerful command
pipelines



11. REDIRECTION

Control input/output

```
echo "Hello" > file.txt
echo "Append" >> file.txt
command < input.txt
command > out.txt 2> err.txt
command >> out.txt 2>&1
command &> all.log
```

Overwrite, append,
redirect error, merge



12. EXPORT, SOURCE, ENV

Manage environment

```
export APP_ENV=prod
source ./config.sh
env | sort
```

source loads variables and
functions from a file



18. AUTOMATION SAFETY

- ✓ Always test in safe environments
- ✓ Validate inputs and variables
- ✓ Use strict mode (set -euxo pipefail)
- ✓ Avoid destructive commands without confirmation
- ✓ Log actions and errors



⚠ DANGEROUS

- rm -rf /
- chmod 777 *
- > /dev/null 2>&1
- curl | bash
- sshpass ...

✓ BETTER PRACTICE

- ✓ Use full paths and confirmations
- ✓ Principle of least privilege
- ✓ Log everything
- ✓ Validate before you change
- ✓ Automate safely



19. WRITING REUSABLE OPERATIONAL SCRIPTS



1. PLAN
Define purpose,
inputs, outputs



2. WRITE
Build script
step-by-step



3. TEST
Test in dev / lab
environments



4. LOG
Add logging +
error handling



5. DOCUMENT
Add usage, help,
examples



6. DEPLOY
Use in automation
pipelines

DOCKER CLI TOOLKIT

CONTROL THE COMPLETE CONTAINER
LIFECYCLE FROM THE TERMINAL



1. CORE DOCKER COMMANDS

COMMAND	PURPOSE	EXAMPLE
<code>docker pull <image></code>	Pull image from registry	<code>docker pull nginx:latest</code>
<code>docker build -t <image> .</code>	Build image from Dockerfile	<code>docker build -t myapp:1.0 .</code>
<code>docker images</code>	List local images	<code>docker images</code>
<code>docker run [options] <image></code>	Run a container	<code>docker run -d -p 8080:80 nginx</code>
<code>docker ps</code>	List running containers	<code>docker ps</code>
<code>docker ps -a</code>	List all containers	<code>docker ps -a</code>
<code>docker exec -it <container> <cmd></code>	Execute command in container	<code>docker exec -it web bash</code>
<code>docker logs <container></code>	View container logs	<code>docker logs -f web</code>
<code>docker inspect <container></code>	Inspect container details	<code>docker inspect web</code>
<code>docker stats <container></code>	Show live resource usage	<code>docker stats web</code>
<code>docker top <container></code>	Show running processes	<code>docker top web</code>
<code>docker cp <src> <container>:<dest></code>	Copy files to/from container	<code>docker cp app.js web:/app</code>
<code>docker stop <container></code>	Stop container gracefully	<code>docker stop web</code>
<code>docker kill <container></code>	Force stop container	<code>docker kill web</code>
<code>docker rm <container></code>	Remove stopped container	<code>docker rm web</code>
<code>docker rmi <image></code>	Remove image	<code>docker rmi myapp:1.0</code>
<code>docker network ls</code>	List networks	<code>docker network ls</code>
<code>docker volume ls</code>	List volumes	<code>docker volume ls</code>
<code>docker system df</code>	Show disk usage summary	<code>docker system df</code>
<code>docker system prune -a</code>	Clean unused data	<code>docker system prune -a</code>
<code>docker history <image></code>	Show image build history	<code>docker history myapp:1.0</code>

2. RUN CONTAINER EXAMPLES

Run with port publishing

```
docker run -d -p 8080:80 --name web nginx
```

Run interactive shell

```
docker run -it --rm ubuntu bash
```

Run with environment variables

```
docker run -d -e APP_ENV=prod -e DEBUG=false myapp:1.0
```

Run with volume mount

```
docker run -v /data/app:/app/data myapp:1.0
```

Run with custom network

```
docker run --network mynet myapp:1.0
```

3. BUILD ARGUMENTS

Pass variables at build time.

Dockerfile

```
ARG APP_VERSION
FROM alpine
ARG APP_VERSION
RUN echo "Building version $APP_VERSION"
```

Build with argument

```
docker build --build-arg APP_VERSION=1.0.0 -t myapp:1.0 .
```

4. ENVIRONMENT VARIABLES

Pass variables at run time.

```
docker run -e ENV=production -e LOG_LEVEL=info myapp:1.0
```

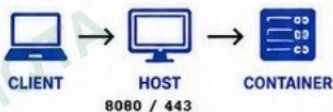
Use --env-file

```
docker run --env-file .env myapp:1.0
```

5. PORT PUBLISHING

Expose container ports to the host.

```
docker run -d -p 8080:80 -p 443:443 nginx
```



6. CONTAINER DEBUGGING

• Get a shell inside a running container

```
docker exec -it <container> bash
```

• Inspect container configuration

```
docker inspect <container>
```

• View logs in real time

```
docker logs -f <container>
```

• Check processes

```
docker top <container>
```

7. IMAGE DEBUGGING

• Inspect image layers

```
docker inspect <image>
```

• View build history

```
docker history <image>
```

• Run a temporary container

```
docker run -it --rm <image> sh
```

• Check image size

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

8. RESOURCE INVESTIGATION

Monitor container and host resources.

• Live stats

```
docker stats
```

• Disk usage

```
docker system df
```

• Detailed usage

```
docker system df -v
```

9. CLEANUP AND DISK USAGE

COMMAND	PURPOSE	EXAMPLE
<code>docker system df</code>	Show disk usage summary	<code>docker system df</code>
<code>docker image prune</code>	Remove dangling images	<code>docker image prune</code>
<code>docker container prune</code>	Remove stopped containers	<code>docker container prune</code>
<code>docker volume prune</code>	Remove unused volumes	<code>docker volume prune</code>
<code>docker network prune</code>	Remove unused networks	<code>docker network prune</code>
<code>docker system prune</code>	Remove unused data	<code>docker system prune</code>
<code>docker system prune -a</code>	Remove all unused data (images, containers, networks, volumes)	<code>docker system prune -a --volumes</code>

10. DEVOPS BEST PRACTICES

- ✓ Use specific image tags, avoid :latest
- ✓ Keep images small and secure
- ✓ Run containers as non-root users
- ✓ Use .dockerignore to reduce build context
- ✓ Scan images for vulnerabilities
- ✓ Clean up unused data regularly
- ✓ Monitor resource usage
- ✓ Automate builds and deployments

QUICK REFERENCE

List containers: `docker ps`
 All containers: `docker ps -a`
 Shell in container: `docker exec -it <c> bash`
 View logs: `docker logs -f <c>`
 Stop container: `docker stop <c>`
 Remove container: `docker rm <c>`
 Remove image: `docker rmi <image>`
 Disk usage: `docker system df`
 Cleanup all: `docker system prune -a`

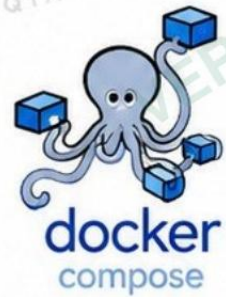
THE CONTAINER LIFECYCLE WITH DOCKER CLI



DOCKER COMPOSE

CLI TOOLKIT

OPERATE MULTI-CONTAINER ENVIRONMENTS



1. DOCKER COMPOSE CORE COMMANDS

COMMAND	PURPOSE	EXAMPLE
<code>docker compose up</code>	Create and start containers	<code>docker compose up -d</code>
<code>docker compose down</code>	Stop and remove containers, networks	<code>docker compose down -v</code>
<code>docker compose ps</code>	List running containers	<code>docker compose ps</code>
<code>docker compose logs</code>	View logs from services	<code>docker compose logs -f web</code>
<code>docker compose exec</code>	Execute command in a service	<code>docker compose exec web bash</code>
<code>docker compose run</code>	Run a one-off command	<code>docker compose run --rm web npm test</code>
<code>docker compose build</code>	Build or rebuild services	<code>docker compose build web</code>
<code>docker compose pull</code>	Pull latest images	<code>docker compose pull</code>
<code>docker compose restart</code>	Restart services	<code>docker compose restart api</code>
<code>docker compose stop</code>	Stop services	<code>docker compose stop</code>
<code>docker compose start</code>	Start stopped services	<code>docker compose start</code>
<code>docker compose config</code>	Validate and view compose file	<code>docker compose config</code>
<code>docker compose images</code>	List images used by services	<code>docker compose images</code>
<code>docker compose top</code>	Display running processes	<code>docker compose top api</code>
<code>docker compose events</code>	View real-time events	<code>docker compose events</code>

2. EXAMPLE: BASIC MULTI-SERVICE APP

docker-compose.yml (sample)

```
version: '3.9'
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - api
  api:
    image: python:3.11-slim
    command: python app.py
    environment:
      - DATABASE_URL=postgres://user:pass@db:5432/app
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - dbdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5
    volumes:
      dbdata:
```

→ STARTUP FLOW



3. KEY FEATURES & HOW TO USE



PROFILES

Enable optional services.

`docker compose --profile debug up`

In compose file:
profiles: ["debug"]



ENVIRONMENT VARIABLES

Use .env file or shell environment.

`docker compose --env-file .env up`

In compose file:
env_file: .env



SERVICE DEPENDENCIES

Control startup order.

```
depends_on:
  db:
    condition: service_healthy
```



NETWORKS

Services communicate over networks.

`docker compose network ls`

Custom network in compose file:
networks:
 backend:



VOLUMES

Persist and share data.

`docker compose volume ls`

Named volume in compose file:
volumes:
 data:



HEALTH CHECKS

Ensure services are healthy.

`docker compose ps`

Check health status column.



4. MULTI-SERVICE TROUBLESHOOTING

PROBLEM	CHECK THIS	COMMANDS
Service not starting	Logs	<code>docker compose logs <service></code>
Service unhealthy	Health check	<code>docker compose ps</code>
Connection failed between services	Service name, network	<code>docker compose exec <svc> sh</code> <code>ping <other-service></code>
Port already in use	Host port conflict	<code>docker compose ps</code> <code>ss -tulpa</code>
Database connection issues	Env vars, depends_on	<code>docker compose config</code> <code>docker compose logs db</code>
Stale containers	Remove & recreate	<code>docker compose down -v</code> <code>docker compose up -d --build</code>
Hanging shutdown	Stop then down	<code>docker compose stop</code> <code>docker compose down</code>

5. COMMON WORKFLOWS

	Start environment → <code>docker compose up -d</code>	Bring up all services in background
	View status → <code>docker compose ps</code>	See running containers
	Follow logs → <code>docker compose logs -f</code>	Stream logs from all services
	Execute command → <code>docker compose exec web bash</code>	Open shell inside a service
	Stop and clean → <code>docker compose down -v</code>	Stop and remove everything



6. BEST PRACTICES

- ✓ Always version-control your compose files.
- ✓ Use .env files for environment-specific values.
- ✓ Define health checks for critical services.
- ✓ Name your services clearly and consistently.
- ✓ Use depends_on with health conditions for reliability.
- ✓ Use docker compose config to validate before deploy.
- ✓ Clean up unused volumes and networks regularly.
- ✓ Document ports, envs, and dependencies.

QUICK REFERENCE

Up (detach):
`docker compose up -d`
Down (remove all):
`docker compose down -v`
Rebuild & Up:
`docker compose up -d --build`
View services:
`docker compose ps`
View logs:
`docker compose logs -f`
Execute in service:

KUBERNETES kubectl TOOLKIT

OPERATE KUBERNETES FROM THE COMMAND LINE



```
>_
```


kubectl

1. RESOURCE OPERATIONS

<code>kubectl get</code>	List resources
<code>kubectl describe</code>	Show detailed info
<code>kubectl apply -f <file></code>	Apply configuration
<code>kubectl create -f <file></code>	Create from file
<code>kubectl delete <resource></code>	Delete a resource
<code>kubectl edit <resource></code>	Edit a resource
<code>kubectl patch <resource></code>	Partially update
<code>kubectl replace -f <file></code>	Replace entire object

EXAMPLES

```
$ kubectl get pods -n dev
$ kubectl describe pod mypod -n dev
$ kubectl apply -f deployment.yaml
```



2. LOGS & EXECUTION

<code>kubectl logs <pod></code>	Show pod logs
<code>kubectl logs -f <pod></code>	Follow logs
<code>kubectl exec -it <pod> -- sh</code>	Open shell in pod
<code>kubectl cp <src> <pod>:/dest</code>	Copy to pod
<code>kubectl cp <pod>:/src <dest></code>	Copy from pod

EXAMPLES

```
$ kubectl logs -f mypod -n dev
$ kubectl exec -it mypod -n dev -- bash
$ kubectl cp ./app.py mypod:/app/app.py
```



3. OBSERVABILITY

<code>kubectl top pods</code>	Pod resource usage
<code>kubectl top nodes</code>	Node resource usage
<code>kubectl get events</code>	List cluster events

EXAMPLES

```
$ kubectl top pods -n dev
$ kubectl top nodes
$ kubectl get events -n dev --sort-by='.1'
```

4. SCALING

<code>kubectl scale deployment <name> --replicas=3</code>	Set replicas
<code>kubectl autoscale deployment <name> \</code> <code>--cpu-percent=70 --min=2 --max=10</code>	Auto scale

EXAMPLES

```
$ kubectl scale deployment web --replicas=3
$ kubectl autoscale deployment web \
--cpu-percent=70 --min=2 --max=10
```



5. ROLLOUT & DEPLOYMENTS

<code>kubectl rollout status deployment/<name></code>	Check rollout status
<code>kubectl rollout history deployment/<name></code>	Show rollout history
<code>kubectl rollout undo deployment/<name></code>	Rollback deployment
<code>kubectl rollout restart deployment/<name></code>	Restart rollout

EXAMPLES

```
$ kubectl rollout status deployment/web
$ kubectl rollout history deployment/web
$ kubectl rollout restart deployment/web
```



6. NETWORKING & ACCESS

<code>kubectl port-forward <pod> 8080:80</code>	Forward local port
<code>kubectl port-forward svc/<service> 8080:80</code>	Forward service port
<code>kubectl run tmp --rm -it --image=busybox -- sh</code>	Temporary pod (test)

EXAMPLES

```
$ kubectl port-forward pod/mypod 8080:80
$ kubectl port-forward svc/web 8080:80
$ kubectl run tmp --rm -it --image=busybox -- sh
```



7. INFORMATION & DISCOVERY

<code>kubectl explain <resource></code>	Help for resource fields
<code>kubectl api-resources</code>	List all API resources
<code>kubectl api-versions</code>	List API versions
<code>kubectl get --raw /</code>	Raw API response

EXAMPLE

```
$ kubectl explain pod.spec.containers
$ kubectl api-resources | grep deploy
$ kubectl api-versions | grep apps
```



8. COMPARISON & VALIDATION

<code>kubectl diff -f <file></code>	Show changes before apply
<code>kubectl wait --for=</code> <code>condition=Ready pod/<name>--timeout=60s</code>	Wait for condition

EXAMPLES

```
$ kubectl diff -f deployment.yaml
$ kubectl wait --for=condition=Ready pod/mypod
--timeout=60s
```



9. AUTHORIZATION

<code>kubectl auth can-i <verb> <resource></code>	Check permission
<code>kubectl auth can-i --list</code>	List all allowed actions

EXAMPLES

```
$ kubectl auth can-i get pods
$ kubectl auth can-i delete deployments -n prod
$ kubectl auth can-i --list -n dev
```



10. DEBUGGING

<code>kubectl debug <pod> -it --image=busybox --</code> <code>--target=<container></code>	Debug running pod
<code>kubectl debug node/<node> -it --image=busybox</code>	Debug node
<code>kubectl get pod <pod> -o yaml</code>	View full YAML
<code>kubectl get pod <pod> -o wide</code>	Wide output
<code>kubectl get pod <pod> -o json</code>	JSON output
<code>kubectl get pod <pod> -o jsonpath='{.status.</code> <code>podIP}'</code>	JSONPath output

EXAMPLES

```
$ kubectl debug pod/mypod -it --image=busybox
--target=mypod
$ kubectl get pod mypod -o jsonpath='{.status.podIP}'
```



CONTEXTS & NAMESPACES

CONTEXTS

- `kubectl config get-contexts`
- `kubectl config use-context <name>`
- `kubectl config current-context`

NAMESPACES

- `kubectl get ns`
- `kubectl get all -n <namespace>`
- Use `-n` or `--namespace` flag



LABELS & SELECTORS

- Label example: `app=web, tier=frontend`
- Select by label: `kubectl get pods -l app=web`
- Multiple: `kubectl get pods -l app=web, tier=frontend`
- Negative: `kubectl get pods -l 'app!=web'`



OUTPUT FORMATS

- `-o yaml` Output in YAML
- `-o json` Output in JSON
- `-o wide` Wide table output

JSONPATH EXAMPLES

<code>{.items[*].metadata.name}</code>	List all names
<code>{.status.podIP}</code>	Pod IP
<code>{.items[*].status.phase}</code>	All pod phases

NETWORK TROUBLESHOOTING WORKFLOW

- 1 IDENTIFY PROBLEM
Application not reachable / slow / failing
- 2 CHECK POD HEALTH
`kubectl get pods -o wide`
- 3 CHECK EVENTS
`kubectl get events --sort-by='.lastTimestamp'`
- 4 CHECK SERVICE
`kubectl get svc / endpoints`
- 5 TEST CONNECTIVITY
`kubectl exec <pod> -- curl <service>`
- 6 CHECK NETWORK
`kubectl get networkpolicies / CNI / DNS`
- 7 CHECK LOGS
`kubectl logs <pod> -c <container>`

DEVOPS FLOW (HIGH LEVEL)

DNS → ROUTE → FIREWALL → PORT → SERVICE → APPLICATION
FOLLOW THIS FLOW TO FIND WHERE THE PROBLEM IS

HELM CLI TOOLKIT



MANAGE KUBERNETES APPLICATIONS AS PACKAGES

>_



1. REPOSITORIES

```
helm repo add <name> <url>
helm repo update
```

Add a chart repository and update index.

EXAMPLE

```
$ helm repo add bitnami \
https://charts.bitnami.com/bitnami
$ helm repo update
```



2. SEARCH CHARTS

```
helm search repo <keyword>
helm search repo <keyword> --versions
```

Search charts in repositories.

EXAMPLE

```
$ helm search repo nginx
$ helm search repo nginx --versions
```



3. INSTALL

```
helm install <release> <chart> [flags]
Install a chart as a release.
```

EXAMPLE

```
$ helm install myapp bitnami/nginx
--namespace dev --create-namespace
```



4. UPGRADE

```
helm upgrade <release> <chart> [flags]
Upgrade an existing release.
```

EXAMPLE

```
$ helm upgrade myapp bitnami/nginx
--namespace dev
--set service.type=NodePort
```



5. ROLLBACK

```
helm rollback <release> <revision>
Rollback to a previous revision.
```

EXAMPLE

```
$ helm rollback myapp 2
$ helm rollback myapp --previous
```



6. UNINSTALL

```
helm uninstall <release>
Remove a release.
```

EXAMPLE

```
$ helm uninstall myapp
--namespace dev
```



7. LIST

```
helm list [--all-namespaces]
List installed releases.
```

EXAMPLE

```
$ helm list -n dev
$ helm list -A
```



8. STATUS

```
helm status <release>
Show release status and details.
```

EXAMPLE

```
$ helm status myapp -n dev
```



9. HISTORY

```
helm history <release>
Show revision history.
```

EXAMPLE

```
$ helm history myapp -n dev
```



10. GET

```
helm get all <release>
helm get values <release>
helm get manifests <release>
helm get notes <release>
Get details about a release.
```

EXAMPLE

```
$ helm get all myapp -n dev
```



11. SHOW

```
helm show chart <chart>
helm show values <chart>
helm show all <chart>
Inspect chart information.
```

EXAMPLE

```
$ helm show values bitnami/nginx
$ helm show all bitnami/nginx
```



12. TEMPLATE

```
helm template <release> <chart> [flags]
Render templates locally.
```

EXAMPLE

```
$ helm template myapp bitnami/nginx \
--values values.yaml
```

KEY HELM CONCEPTS



VALUES FILES

Customize chart configuration using YAML files. Pass with -f values.yaml



--SET

Override values from CLI using --set key=value. Example: --set image.tag=1.2.3



NAMESPACES

Helm releases are scoped to namespaces. Use -n <namespace>



RELEASE MANAGEMENT

Each install/upgrade creates a release with a revision history.



CHART INSPECTION

Use helm show, helm get, helm template to explore charts before deployment.



TEMPLATE RENDERING

Render output locally with helm template to preview Kubernetes manifests.

HELM UPGRADE WORKFLOW



INSPECT CHART



RENDER TEMPLATE



SET VALUES (&/OR -f FILE)



UPGRADE RELEASE



VERIFY STATUS

```
$ helm show values <chart> > values.yaml
$ helm template myapp <chart> -f values.yaml
$ helm upgrade myapp <chart> -f values.yaml --namespace dev
$ helm status myapp -n dev
```

HELM ROLLBACK WORKFLOW



CHECK HISTORY



CHOOSE REVISION



ROLLBACK RELEASE



VERIFY STATUS

```
$ helm history myapp -n dev
$ helm rollback myapp 3 -n dev
$ helm status myapp -n dev
```

UPGRADE TROUBLESHOOTING

- Check release status: `helm status <release>`
- Review history: `helm history <release>`
- Render templates locally: `helm template ...`
- Validate values: `helm lint <chart>`
- Check events: `kubectl get events -n <ns>`
- Review logs of failing pods
- Use --atomic to rollback on failure
- Use --dry-run --debug for detailed output

AWS CLI TOOLKIT

OPERATE AWS INFRASTRUCTURE FROM THE TERMINAL



>_

1. SETUP & AUTHENTICATION

<code>aws configure</code>	Configure access keys,
<code>aws configure list</code>	Show current configuration
<code>aws configure set</code>	Set config values
<code>aws sts get-caller-identity</code>	Verify identity
<code>aws sts get-session-token</code>	Get temporary credentials

EXAMPLE

```
$ aws configure --profile dev
$ aws sts get-caller-identity --profile dev
$ aws configure list
```

2. CORE SERVICES

<code>aws ec2</code>	EC2 instances & networking
<code>aws s3</code>	S3 object storage
<code>aws s3api</code>	Low-level S3 API operations
<code>aws iam</code>	IAM users, roles, policies
<code>aws eks</code>	EKS cluster management
<code>aws ecr</code>	ECR container registry
<code>aws elbv2</code>	Elastic Load Balancers (ALB/NLB)
<code>aws autoscaling</code>	Auto Scaling groups & policies
<code>aws rds</code>	Relational database service
<code>aws route53</code>	DNS & domain management
<code>aws cloudwatch</code>	Metrics, alarms, dashboards
<code>aws logs</code>	CloudWatch Logs
<code>aws secretsmanager</code>	Secrets management
<code>aws ssm</code>	Systems Manager operations

3. PROFILES & REGIONS

<code>--profile <name></code>	Use named profile
<code>--region <region></code>	Specify AWS region
<code>aws configure list-profiles</code>	List all profiles
<code>aws configure get region</code>	Show current region
<code>AWS_DEFAULT_PROFILE</code>	Set default profile
<code>AWS_DEFAULT_REGION</code>	Set default region

EXAMPLE

```
$ aws ec2 describe-instances --profile dev
--region us-east-1
$ AWS_DEFAULT_PROFILE=prod aws s3 ls
```

4. ADVANCED CLI OPTIONS

<code>--query</code>	Filter results with JMESPath
<code>--filter</code>	Filter results (service specific)
<code>--output</code>	Output format: json, text, table, yaml
<code>--max-items</code>	Limit number of results
<code>--no-paginate</code>	Disable pagination
<code>--debug</code>	Verbose debug output

EXAMPLE

```
$ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].(ID:InstanceId,
State:State.Name,IP:PublicIpAddress)" \
--output table --region us-east-1
```

5. JMESPATH QUICK REFERENCE

<code>Reservations[*]</code>	All reservations
<code>Instances[*]</code>	All instances
<code>[].InstanceId</code>	Instance IDs
<code>[].(ID:InstanceId,IP:PublicIpAddress)</code>	Custom output
<code>[?State.Name=="running"]</code>	Filter expression
<code>sort_by(@,&InstanceId)</code>	Sort results
<code>length(@)</code>	Count items

EXAMPLE

```
$ aws ec2 describe-instances \
--query "Reservations[].Instances[?State.
Name=='running'].InstanceId" --output text
```

6. ASSUME-ROLE WORKFLOW

Create IAM role	In target account
Attach trust policy	Allow trusted account
Use sts assume-role	Get temporary credentials
Export credentials	Use in CLI commands
Set session duration	Control credential lifetime

EXAMPLE

```
$ aws sts assume-role \
--role-arn arn:aws:iam::123456789012:role/Admin \
--role-session-name dev-session --profile dev
# Export the returned AccessKeyId, SecretAccessKey,
and SessionToken as environment variables
```

7. INFRASTRUCTURE INSPECTION

<code>aws ec2 describe-instances</code>	List EC2 instances
<code>aws ec2 describe-security-groups</code>	List security groups
<code>aws ec2 describe-vpcs</code>	List VPCs
<code>aws ec2 describe-subnets</code>	List subnets
<code>aws s3 ls</code>	List S3 buckets
<code>aws iam list-users</code>	List IAM users
<code>aws iam list-roles</code>	List IAM roles
<code>aws rds describe-db-instances</code>	List RDS databases
<code>aws eks list-clusters</code>	List EKS clusters

EXAMPLE

```
$ aws ec2 describe-instances --query
"Reservations[*].Instances[*].(ID:InstanceId,Type:InstanceType,State:State.Name)" --output table
```

8. IDENTITY TROUBLESHOOTING

<code>aws sts get-caller-identity</code>	Verify current identity
<code>aws iam get-user</code>	Current user details
<code>aws iam list-attached-user-policies</code>	User policies
<code>aws iam simulate-principal-policy</code>	Test permissions
<code>aws iam list-role-policies</code>	Role policies
<code>aws iam get-role</code>	Role details

EXAMPLE

```
$ aws sts get-caller-identity
$ aws iam simulate-principal-policy \
--policy-source-arn arn:aws:iam::123456789012:user/DevUser \
--action-names s3:ListBucket s3:GetObject \
--resource-arns arn:aws:s3:::my-bucket arn:aws:s3:::my-bucket/*
```

9. CLOUDWATCH LOG RETRIEVAL

<code>aws logs describe-log-groups</code>	List log groups
<code>aws logs describe-log-streams</code>	List log streams
<code>aws logs get-log-events</code>	Get log events
<code>aws logs filter-log-events</code>	Filter logs by pattern
<code>aws logs tail</code>	Tail log stream (CLI v2)

EXAMPLE

```
$ aws logs filter-log-events \
--log-group-name /aws/eks/my-cluster/cluster \
--start-time $(date -d '1 hour ago' +%s)000 \
--filter-pattern "ERROR" \
--output table
```

10. COMMON OUTPUT FORMATS

<code>{ }</code> json Raw JSON output <code>--output json</code>	<code>table</code> Human-readable in tables <code>--output table</code>	<code>T</code> text Plain text output <code>--output text</code>	<code>YAML</code> yaml YAML output <code>--output yaml</code>	<code>YAML stream</code> YAML stream (multi-doc) <code>--output yaml-stream</code>
---	---	---	--	--

DEVOPS AWS CLI WORKFLOW



BEST PRACTICES

- Never hardcode access keys.
- Use IAM roles & least privilege.
- Always specify `--region`.
- Use `--profile` for multiple accounts.
- Test with `--dry-run` where available.

CLI SAFETY TIPS

- Review `--query` output before using in scripts.
- Use `--no-cli-pager` for automation.
- Avoid destructive commands without confirmation.
- Use CloudWatch alarms for critical resources.

HANDY SHORTCUTS

- Press TAB for auto-completion.
- Press `↑` for command history.
- Use `aws help` for global help.
- Use `aws <service> help` for service help.

LEARN MORE

Official Docs

TERRAFORM CLI TOOLKIT



Terraform



MANAGE INFRASTRUCTURE AS CODE FROM THE COMMAND LINE



1. INITIALIZATION

terraform init

Initialize a Terraform working directory.

EXAMPLE

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
```



2. CODE QUALITY & VALIDATION

terraform fmt
terraform validate

Format configuration files
Validate configuration

EXAMPLE

```
$ terraform fmt -recursive
$ terraform validate
```



3. PLAN

terraform plan

Preview changes before applying.

EXAMPLE

```
$ terraform plan
$ terraform plan -out=tfplan
```



4. APPLY

terraform apply [planfile]

Apply changes to reach desired state.

EXAMPLE

```
$ terraform apply
$ terraform apply tfplan
```



5. DESTROY

terraform destroy

Destroy managed infrastructure.

EXAMPLE

```
$ terraform destroy
$ terraform destroy -auto-approve
```



6. INSPECT & OUTPUT

terraform show

Show human-readable plan or state

terraform output

Show output values

terraform providers

List required providers

EXAMPLE

```
$ terraform show tfplan
$ terraform output
$ terraform providers
```



7. INTERACTIVE & VISUAL

terraform console

Interactive REPL for expressions

terraform graph

Generate dependency graph

EXAMPLE

```
$ terraform console
> aws_s3_bucket.example.id
$ terraform graph | dot -Tpng graph.png
```



8. IMPORT EXISTING RESOURCES

terraform import <address> <id>

Bring existing infrastructure under Terraform management.

EXAMPLE

```
$ terraform import
aws_s3_bucket.example my-existing-bucket
```



9. STATE MANAGEMENT COMMANDS

terraform state list

List resources in the state.

terraform state show

Show details of a resource.

terraform state mv

Move/rename a resource in the state.

terraform state rm

Remove a resource from the state (not from infra).

terraform force-unlock

Unlock a stuck state.

EXAMPLE

```
$ terraform state list
```

EXAMPLE

```
$ terraform state show
aws_s3_bucket.example
```

EXAMPLE

```
$ terraform state mv
aws_instance.old
aws_instance.new
```

EXAMPLE

```
$ terraform state rm
aws_instance.old
```

EXAMPLE

```
$ terraform force-unlock
<LOCK_ID>
```



10. WORKSPACES

terraform workspace list

List all workspaces

terraform workspace new <name>

Create a new workspace

terraform workspace select <name>

Switch workspace

terraform workspace delete <name>

Delete a workspace

terraform workspace show

Show current workspace

EXAMPLE

```
$ terraform workspace list
$ terraform workspace new dev
$ terraform workspace select dev
```



11. PLAN FILES

terraform plan -out=tfplan

Save plan to file

terraform show tfplan

Inspect saved plan

terraform apply tfplan

Apply saved plan

EXAMPLE

```
$ terraform plan -out=tfplan
$ terraform show tfplan
$ terraform apply tfplan
```



12. TARGETING

terraform plan -target=addr
terraform apply -target=addr

Apply changes to specific resource(s) only.

EXAMPLE

```
$ terraform plan -target=
aws_instance.web
$ terraform apply -target=
aws_instance.web
```



13. DRIFT DETECTION

terraform plan

Detect changes between real infrastructure and state.

EXAMPLE

```
$ terraform plan
No changes.
(or shows drifted resources)
```

CORE CONCEPTS



STATE

Terraform stores the mapping of real resources in the state file.



STATE LOCKING

Prevents concurrent operations that can corrupt the state.



WORKSPACES

Isolated environments within the same state for dev, test, prod, etc.



PLAN BEFORE APPLY

Always review the plan. No surprises in production.



IMPORT

Adopt existing resources into Terraform—do not recreate.



TARGETING

Use only for emergencies or specific fixes.



DRIFT DETECTION

Run plan regularly to detect out-of-band changes.

PRODUCTION-SAFE WORKFLOW



WRITE CODE



FMT & VALIDATE



PLAN



REVIEW



APPLY



VERIFY

- ✓ Always run terraform fmt and terraform validate
- ✓ Always run terraform plan and review carefully
- ✓ Use plan files for approvals in production

- ✓ Never apply without understanding the changes
- ✓ Monitor state locking and avoid manual state edits
- ✓ Backup state before major operations

COMMON BEST PRACTICES

- ✓ Store state remotely (S3 + DynamoDB)
- ✓ Enable state locking
- ✓ Use workspaces for environments
- ✓ Use meaningful resource names
- ✓ Keep state secure and backed up
- ✓ Avoid manual changes in the console
- ✓ Import, do not recreate
- ✓ Review plans in CI/CD pipelines
- ✓ Tag resources
- ✓ Document

ANSIBLE CLI TOOLKIT

AUTOMATE CONFIGURATION AND SERVER MANAGEMENT



> 1. CORE ANSIBLE CLI COMMANDS

ansible	- Run ad hoc commands
ansible-playbook	- Run playbooks
ansible-inventory	- Manage inventory
ansible-config	- View configuration
ansible-doc	- Get module documentation
ansible-galaxy	- Manage roles and collections
ansible-vault	- Encrypt and decrypt data

> 2. AD HOC COMMAND EXAMPLES

ansible all -m ping	- Check connectivity
ansible web -m shell -a "uptime"	- Run command on hosts
ansible db -m service -a "name=nginx state=started"	- Start service
ansible all -m copy -a "src=/tmp/file dest=/tmp/"	- Copy file
ansible all -m yum -a "name=httpd state=installed"	- Install package

3. INVENTORY TARGETING

Inventory file (hosts.ini)

```
[web]
web1.example.com
web2.example.com

[db]
db1.example.com
db2.example.com

[all:children]
web
db
```

Common targets

all	- All hosts
web	- Group
web1	- Host
web:db	- Multiple groups
web:&staging	- Intersection
web:!db	- Exclude group
web[0]	- First host in group
web[1:3]	- Host range

Use -i to specify inventory file
ansible all -i hosts.ini -m ping

4. HELPFUL CLI OPTIONS

--limit "pattern"	- Limit hosts (e.g., --limit web)
--tags "tag"	- Run tasks with tag
--skip-tags "tag"	- Skip tasks with tag
--check	- Dry run (no changes)
--diff	- Show changes in detail
--list-hosts	- List hosts matched
--list-tasks	- List tasks in playbook
--syntax-check	- Check playbook syntax
-v	- Increase verbosity (use -vvv)

5. ANSIBLE PLAYBOOK EXAMPLE

```
---
- name: Install and start Nginx
  hosts: web
  become: yes
  vars:
    nginx_port: 80
  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present
    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

Run the playbook
ansible-playbook site.yml

Dry run (safe)
ansible-playbook site.yml --check

Show changes
ansible-playbook site.yml --diff

6. VARIABLES AND VAULT SECRETS

Variables (vars.yml)

```
app_user: devops
app_port: 8080
```

Use variables

ansible-playbook site.yml -e "app_port=9090"

Vault secrets

ansible-vault encrypt secrets.yml

Use vault file

ansible-playbook site.yml --ask-vault-pass

Create encrypted string

ansible-vault encrypt_string 'mypassword' --name 'db_pass'

7. TROUBLESHOOTING & VERBOSITY

Increase verbosity

-v	Basic output
-vv	More details
-vvv	Debug output

Common troubleshooting

• SSH connection failed	→ Check SSH access & keys
• Permission denied	→ Check become/sudo config
• Host unreachable	→ Check inventory & network
• Module not found	→ Check Ansible version/collections
• Task failed	→ Use -vvv for more details

8. SAFE CONFIGURATION ROLLOUT

1. Validate inventory	→ ansible-inventory --list
2. Test connectivity	→ ansible all -m ping
3. Dry run first	→ ansible-playbook site.yml --check
4. Review changes	→ ansible-playbook site.yml --diff
5. Run on subset	→ --limit "group"
6. Rollout to all	→ ansible-playbook site.yml

DEVOPS TIP

Always test with --check and --diff before applying changes. Small steps prevent big outages.



WARNING

Ad hoc commands are powerful. Double-check modules and options to avoid unintended changes.

</> PRACTICE THIS

1. Run **ansible all -m ping**
2. Create a simple playbook and test with
3. Try **--tags**, **--limit**, and **--diff**

> ANSIBLE + CLI = AUTOMATION, CONSISTENCY, AND RELIABILITY

JENKINS CLI AND PIPELINE TOOLKIT

OPERATE AND TROUBLESHOOT JENKINS FROM THE COMMAND LINE



Jenkins

1. JENKINS CLI CLIENT

Jenkins CLI lets you manage Jenkins from your terminal.

Download:

<https://www.jenkins.io/download/cli/>

Install (example)

```
curl -O https://repo.jenkins-ci.org/public/org/jenkins-ci/main/jenkins-cli/2.440/jenkins-cli-2.440.jar
java -jar jenkins-cli-2.440.jar -s http://jenkins.example.com/ help
```

2. AUTHENTICATION

Use API token (recommended)

1. Go to: **Manage Jenkins > Security > API Tokens**
2. Create a new token
3. Use it with CLI or curl

CLI login

```
java -jar jenkins-cli.jar -s http://jenkins.example.com/ login --username devops --password-stdin <<< "YOUR_API_TOKEN"
```

3. JOB OPERATIONS (CLI)

- | | |
|-------------|------------------------------|
| list-jobs | - List all jobs |
| get-job | - Get job details (XML) |
| build | - Trigger a build |
| build-job | - Trigger build with options |
| copy-job | - Copy a job |
| delete-job | - Delete a job |
| disable-job | - Disable a job |
| enable-job | - Enable a job |
| rename-job | - Rename a job |

4. BUILD STATUS & HISTORY

- | | |
|-------------------|----------------------------|
| queue-info | - View build queue |
| who-am-i | - Show current user |
| builds | - Show builds for a job |
| build-history | - History with details |
| last-build | - Get last completed build |
| next-build-number | - Next build number |
| current-build | - Info about current build |
| list-views | - List all views |
| view-job | - List jobs in a view |

5. CONSOLE OUTPUT & LOGS

- | | |
|-----------------------------------|-----------------------------|
| console <job> <build#> | - Get console log |
| console-tail <job> <build#> | - Tail console log |
| console-tail -f <job> <build#> | - Follow log live |
| get-build <job> <build#> | - Get build details (XML) |
| get-build-env-vars <job> <build#> | - Get environment variables |

6. NODE & PLUGIN INSPECTION

- | | |
|----------------------|----------------------------|
| list-nodes | - List all nodes/agents |
| node-info <node> | - Get node details |
| computer-info <node> | - Get computer info |
| list-plugins | - List installed plugins |
| get-plugin <name> | - Get plugin details (XML) |
| safe-restart | - Restart Jenkins safely |
| system-info | - System information |

7. JENKINS REST API WITH CURL

All Jenkins features are available over HTTP API. Base URL: <http://jenkins.example.com> with your API token.

```
# Example: Get all jobs (JSON)
curl -u devops:YOUR_API_TOKEN \
"http://jenkins.example.com/api/json?tree=jobs[name,url]"

# Example: Get build info
curl -u devops:YOUR_API_TOKEN \
"http://jenkins.example.com/job/my-job/123/api/json"
```

8. TRIGGER BUILDS THROUGH API

Trigger a build

```
curl -u devops:YOUR_API_TOKEN -X POST http://jenkins.example.com/job/my-job/build
```

Trigger with parameters

```
curl -u devops:YOUR_API_TOKEN -X POST --data "PARAM1=value1&PARAM2=value2" http://jenkins.example.com/job/my-job/buildWithParameters
```

9. RETRIEVE BUILD RESULTS

Check if build is building, success or failed

```
curl -u devops:YOUR_API_TOKEN \
"http://jenkins.example.com/job/my-job/123/api/json?tree=result"
```

Possible results:

SUCCESS, FAILURE, UNSTABLE, ABORTED, NOT_BUILT, IN_PROGRESS

10. PIPELINE LOGS & ENVIRONMENT

Pipeline console log

```
curl -u devops:YOUR_API_TOKEN \
"http://jenkins.example.com/job/my-pipeline/45/consoleText"
```

Pipeline environment variables

```
curl -u devops:YOUR_API_TOKEN \
"http://jenkins.example.com/job/my-pipeline/45/api/json?tree=actions[environment[*]]"
```

11. CREDENTIALS AWARENESS

- ✓ Never expose credentials in logs
- ✓ Use Jenkins Credentials Store
- ✓ Use withCredentials in pipelines
- ✓ Use Secret Text / Secret File
- ✓ Avoid printing environment variables
- ✓ Mask secrets in console output
- ✓ Use API tokens instead of passwords

12. AGENT & WORKSPACE TROUBLESHOOTING

Agent troubleshooting

```
java -jar jenkins-cli.jar -s http://jenkins.example.com/ list-nodes
java -jar jenkins-cli.jar -s http://jenkins.example.com/ node-info <node>
```

Workspace troubleshooting

```
rm -rf $WORKSPACE/* # Clean workspace (careful)
du -sh $WORKSPACE # Check workspace size
df -h # Disk space on agent
```

13. FAILED-STAGE INVESTIGATION

- | | |
|----------------------------------|------------------------------------|
| 1. Get console log | 7. Check credentials & permissions |
| 2. Check build status | 8. Check agent resource usage |
| 3. Inspect environment variables | 9. Check network connectivity |
| 4. Check workspace files | 10. Reproduce the issue locally |
| 5. Review recent changes | |
| 6. Check external dependencies | |



Investigate before you fix.
Observe > Understand > Fix

14. CI/CD AUTOMATION FROM SCRIPTS

Jenkins CLI + API = Power

Use CLI or API in:

- Deployment scripts
- Release automation
- Build orchestration
- Monitoring and alerting
- Infrastructure pipelines

Example: Trigger build and wait

```
curl -u devops:TOKEN -X POST http://jenkins/job/app/build
BUILD=123
while true; do
  RESULT=$(curl -s -u devops:TOKEN \
    "http://jenkins/job/app/$BUILD/api/json?tree=result" | jq -r '.result')
  [ "$RESULT" != "null" ] && break
  sleep 5
done
echo "Build $BUILD finished with $RESULT"
```



DEVOPS TIP

Automate everything you can.
Observe everything you change.
Document everything you automate.



SECURITY REMINDER

Protect your API tokens.
Keep credentials secret.
Audit access regularly.



REMEMBER

The command line is your fastest
to visibility, automation, and control.
Use it wisely.

ARGO CD CLI TOOLKIT

OPERATE GITOPS DEPLOYMENTS FROM THE TERMINAL



1. CORE ARGO CD CLI COMMANDS

<code>argocd login</code>	- Login to Argo CD server
<code>argocd account</code>	- Manage account settings
<code>argocd cluster</code>	- Manage clusters
<code>argocd repo</code>	- Manage repositories
<code>argocd app</code>	- Manage applications

2. APPLICATION MANAGEMENT COMMANDS

<code>argocd app create</code>	- Create a new application
<code>argocd app get</code>	- Get details of an application
<code>argocd app list</code>	- List all applications
<code>argocd app diff</code>	- Show difference (drift) between desired & live
<code>argocd app sync</code>	- Synchronize application to desired state
<code>argocd app wait</code>	- Wait for application to reach desired state
<code>argocd app history</code>	- Show revision history of an application
<code>argocd app rollback</code>	- Rollback application to a previous revision
<code>argocd app logs</code>	- Get application controller logs
<code>argocd app resources</code>	- List live resources managed by the application

3. LOGIN EXAMPLE

```
argocd login argocd.example.com
--username admin
--password
--insecure
```

Or use token:

```
argocd login argocd.example.com
--auth-token $ARGOCD_AUTH_TOKEN
```

4. VIEWING APPLICATIONS

```
argocd app list -o wide
argocd app get my-app
argocd app get my-app -o yaml
```

5. SYNC OPERATIONS

```
argocd app sync my-app
argocd app sync my-app --prune --force
```

- ✔ `--prune` Removes resources no longer defined in Git
- ✔ `--force` Forces sync even if resources are unchanged

6. WAIT FOR SYNC

```
argocd app wait my-app
--health --sync --timeout 300
```

Waits until the application is Healthy and Synced.

7. HISTORY & ROLLBACK

```
argocd app history my-app
argocd app rollback my-app <REVISION_ID>
```

Rollback reverts the application to a previous Git revision.

8. STATUS & HEALTH

```
argocd app get my-app -o wide
```

Shows SYNC STATUS and HEALTH STATUS.

SYNC STATUS:

- Synced
- OutOfSync
- Unknown / Error

HEALTH STATUS:

- Healthy
- Degraded
- Unhealthy
- Missing

9. RESOURCES & LOGS

```
argocd app resources my-app
```

```
argocd app logs my-app
--follow
```

View all Kubernetes resources managed by the application.
Follow application controller logs.

10. GITOPS STATUS INSIGHTS

- ✔ Sync status shows if live state matches Git
- ✔ Health status shows runtime health of resources
- ✔ Application drift appears as OutOfSync
- ✔ Manual sync pushes changes from Git to cluster
- ✔ Automated sync keeps cluster aligned with Git
- ✔ Rollbacks restore a previous known good state
- ✔ Logs and events help troubleshoot deployment issues

11. GITOPS TROUBLESHOOTING

- ✔ Check app status: `argocd app get <app>`
- ✔ View differences: `argocd app diff <app>`
- ✔ Check sync history: `argocd app history <app>`
- ✔ Inspect live resources: `argocd app resources <app>`
- ✔ Check controller logs: `argocd app logs <app>`
- ✔ Verify cluster: `argocd cluster list`
- ✔ Verify repository: `argocd repo list`

12. DEPLOYMENT ROLLBACK FLOW

- 1 Check history
`argocd app history <app>`
- 2 Identify revision
Choose a known good revision
- 3 Rollback
`argocd app rollback <app> <revision>`
- 4 Wait for healthy state
`argocd app wait <app> --health`
- 5 Verify
`argocd app get <app> -o wide`



QUICK TIP

Always commit to Git first. Argo CD will do the safe, repeatable deployment for you.
Git = Source of Truth | Argo CD = Automation Engine | Kubernetes = Runtime

PROMETHEUS CLI AND PROMQL TOOLKIT



Prometheus

INVESTIGATE METRICS WITHOUT RELYING ONLY ON DASHBOARDS.

1. PROMETHEUS HTTP API BASICS

Prometheus exposes a HTTP API for querying metrics and scraping data.

Endpoint	Purpose
/api/v1/query	Instant query (single point in time)
/api/v1/query_range	Range query (over a time window)
/targets	List all scrape targets and status
/metrics	Raw metrics exposed by Prometheus
/api/v1/alerts	List active alerts

2. CURL AGAINST PROMETHEUS

Set your Prometheus base URL

```
PROM_URL=http://prometheus:9090
```

Check Prometheus status

```
curl -s $PROM_URL/-/healthy
```

List all targets

```
curl -s $PROM_URL/targets
```

Get metric names

```
curl -s $PROM_URL/metrics | head
```

3. INSTANT & RANGE QUERIES

Instant Query (now)

```
curl -G $PROM_URL/api/v1/query \
--data-urlencode 'query=up'
```

Range Query (last 5m, 15s step)

```
curl -G $PROM_URL/api/v1/query_range \
--data-urlencode 'query=rate(http_requests_total[5m])' \
--data-urlencode 'start=$(date -d "5 min ago" +%s)' \
--data-urlencode 'end=$(date +%s)' \
--data-urlencode 'step=15s'
```

Response is JSON

Use jq to extract results

```
curl -G $PROM_URL/api/v1/query --data-urlencode \
'query=up' | jq '.data.result'
```

4. PROMQL FUNDAMENTALS

PromQL is Prometheus Query Language.

Concept	Example
Metric name	http_requests_total
Label selector	http_requests_total{job="api"}
Multiple labels	http_requests_total{job="api",env="prod"}
Regex match	http_requests_total{instance=~"10\.0\.\.*"}
Negative match	http_requests_total{job!="batch"}
Filter by value	http_requests_total > 100
Filter by label value	http_requests_total{status="500"}

5. FILTERING & OPERATORS

Operations

Example

Equal	up == 1
Not equal	up != 1
Greater than	http_requests_total > 100
Less than	http_requests_total < 100
Regex match	job =~ "api web"
Regex not match	job !~ "test dev"
Exists	http_requests_total{job!=""}
Logical AND	up == 1 and on(instance) job="api"
Logical OR	up == 0 or job="api"
Logical UNLESS	up unless job="blackbox"

6. AGGREGATION FUNCTIONS

Aggregate across dimensions.

Function

Example

rate()	rate(http_requests_total[5m])
increase()	increase(http_requests_total[1h])
sum()	sum(rate(http_requests_total[5m]))
avg()	avg(cpu_usage_seconds_total)
min()	min(node_memory_MemFree)
max()	max(node_cpu_seconds_total)
count()	count(up)
count_values()	count_values("status", http_requests_total)
histogram_quantile()	histogram_quantile(0.95, rate(request_duration_seconds_bucket[5m]))

Use by() or without() to control grouping.

Example: sum(rate(http_requests_total[5m])) by (job, instance)

7. USEFUL PROMQL SNIPPETS

CPU usage (per instance)	100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Memory usage %	100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
Top 5 CPU consumers	topk(5, sum(rate(container_cpu_usage_seconds_total[5m])) by (pod, namespace))
Top 5 memory consumers	topk(5, sum(container_memory_working_set_bytes) by (pod, namespace))
Error rate (5xx)	sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
Request rate	sum(rate(http_requests_total[5m])) by (job)
Pod restart rate	sum(rate(kube_pod_container_status_restarts_total[5m])) by (pod, namespace)

8. TARGET INSPECTION & SERVICE DISCOVERY

List all targets

```
curl -s $PROM_URL/targets | jq '.data.activeTargets[] | \
{job: .labels.job, instance: .labels.instance, health: .health}'
```

Targets summary

```
curl -s $PROM_URL/targets | jq '.data.activeTargets | length'
```

Service discovery configs

```
curl -s $PROM_URL/api/v1/status/config | jq \
'.data.configScrapeConfigs[] | job_name'
```

Check scrape config for a job

```
curl -s $PROM_URL/api/v1/status/config | \
jq --arg job "api-server" \
'.data.configScrapeConfigs[] | select(.job_name==$job)'
```

9. PROMTOOL UTILITIES

Check Prometheus config	promtool check config prometheus.yml
Check rule files	promtool check rules rules.yml
Test rules (unit test)	promtool test rules test_rules.yml
Check TSDB	promtool tsdb analyze /path/to/data

10. QUERY DEBUGGING & ALERT INVESTIGATION

- Start simple, then add filters
- Use up to verify scraping
- Use count() to understand cardinality
- Use sum by(job) to see distribution
- Check /targets for unhealthy targets
- Validate rules with promtool before deploying
- Investigate alerts via /api/v1/alerts
- Correlate alerts with logs and events

11. GITOPS & ALERTING BEST PRACTICES

- Store alerts & rules in Git
- Validate with promtool in CI/CD
- Use labels (team, env, service) consistently
- Keep alerts actionable and specific
- Use recording rules for heavy queries
- Document runbooks for each alert

12. REAL DEVOPS WORKFLOW (CLI FIRST)



1. Observe Alert / Symptom



2. Inspect Targets /targets



3. Query Metrics /query or /query_range



4. Filter & Aggregate PromQL



5. Investigate Alert /alerts + Logs



GRAFANA AND LOKI CLI TOOLKIT

INVESTIGATE DASHBOARDS, LOGS, AND
OBSERVABILITY SYSTEMS FROM THE TERMINAL.



GRAFANA

Interact with Grafana using
its powerful HTTP API.



1. GRAFANA HTTP API OVERVIEW

Method	Endpoint	Purpose
GET	/api/health	Health check
GET	/api/dashboards	List dashboards
GET	/api/dashboards/uid/uid	Get dashboard
GET	/api/datasources	List data sources
GET	/api/folders	List folders
GET	/api/alerts	List alerts
POST	/api/serviceaccounts	Create service account
GET	/api/serviceaccounts/id/tokens	Create token
GET	/api/search	Global search



2. CURL AUTHENTICATION

Basic Auth (user:admin)

```
curl -u admin:admin http://grafana:3000/api/health
```

API Token Auth (recommended)

```
curl -H "Authorization: Bearer $GRAFANA_TOKEN" \
http://grafana:3000/api/health
```

Service Account Token

```
curl -H "Authorization: Bearer $SA_TOKEN" \
http://grafana:3000/api/dashboards
```



3. HEALTH API

Check Grafana health

```
curl http://grafana:3000/
api/health
```

Response: ("database":"ok",
"version":"x.x.x", "commit":"...")



4. DASHBOARD API

List all dashboards

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/dashboards
```

Get dashboard by UID

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/dashboards/uid/abc123
```



5. DATA SOURCE API

List data sources

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/datasources
```

Get a data source

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/datasources/1
```



6. FOLDER API

List folders

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/folders
```



7. ALERTING API

List all alert rules

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/ruler/grafana/api/v1/rules
```



8. SERVICE ACCOUNT TOKENS

Create service account

```
curl -H "Authorization: Bearer $TOKEN" -X POST http://grafana:3000/
api/serviceaccounts --data '{"name":"cli"}'
```

Create token

```
curl -H "Authorization: Bearer $TOKEN" -X POST http://grafana:3000/
api/serviceaccounts/1/tokens --data '{"name":"cli-token"}'
```



9. DASHBOARD RETRIEVAL

Export dashboard JSON

```
curl -H "Authorization: Bearer $TOKEN" http://grafana:3000/
api/dashboards/uid/abc123
| jq -r '.dashboard' > dashboard.json
```



10. API TROUBLESHOOTING

- ✓ Unauthorized (401): Check token
- ✓ Forbidden (403): Check permissions
- ✓ Not Found (404): Check endpoint
- ✓ Use /api/health first
- ✓ Inspect response body with jq
- ✓ Verify token scope & role
- ✓ Check Grafana server logs



LOKI

Query and investigate logs
at scale using LogQL
and Loki APIs.



1. LOGCLI BASICS

Install logcli

```
brew install grafana/tap/logcli
```

or

```
wget https://github.com/grafana/
loki/releases/latest/download/
logcli-linux-amd64.zip
```



2. LOGCLI EXAMPLES

Query logs

```
logcli query '{job="app"}'
```

Tail logs

```
logcli tail '{job="app"}'
```

Tail with follow

```
logcli tail -f '{job="app"}'
```



3. LOKI HTTP API

List label names

```
curl -G http://loki:3100/loki/api/v1/
labels
```

List label values

```
curl -G "http://loki:3100/loki/api/v1/
label/job/values"
```

Query logs

```
curl -G --data-urlencode 'query=
{job="app"}' http://loki:3100/loki/
api/v1/query
```



4. LOGQL FUNDAMENTALS

- ✓ Label selector: {job="app"}
- ✓ Multiple selector: {job="app", env="prod"}
- ✓ Regex selector: (namespace=~"prod.*")
- ✓ Negative selector: {job!="batch"}
- ✓ Line filter: {job="app"} != "error"
- ✓ Regex filter: {job="app"} =~ "exception|failed"
- ✓ Exclusion filter: {job="app"} != "debug"



5. STREAM SELECTORS

Select specific log streams

```
{job="payment", namespace="prod"}
```



6. FILTERING LOGS

Filter log lines

```
{job="payment"} != "timeout"
```

Exclude lines

```
{job="payment"} != "healthz"
```



7. TIME RANGES

Last 5 minutes

```
{job="app"} =~ "error" | line_format ""
--since 5m
```

Custom range

```
logcli query '{job="app"}' --since 1h --until 10h
```



8. QUERY LIMITS

Limit number of returned logs

```
logcli query '{job="app"}' --limit 1000
```

Increase limit

```
logcli query '{job="app"}' --limit 5000
```



9. KUBERNETES LOG INVESTIGATION

Query by Kubernetes labels

```
{namespace="prod", pod=~"api.*"}
```

Check container logs

```
{container="app"} != "error"
```

10. CORRELATION WORKFLOW



1. METRIC SPIKE
Detect issue in
Prometheus



2. DASHBOARD
Investigate metrics
in Grafana



3. LOKI QUERY
Query logs with
LogQL / logcli



4. APPLICATION LOG
Find errors and
patterns



5. ROOT CAUSE
Identify, fix, and
prevent recurrence

11. BEST PRACTICES

- ✓ Use Service Account Tokens for automation
- ✓ Store tokens securely (Vault / Secrets Manager)
- ✓ Limit token permissions
- ✓ Log and monitor API usage
- ✓ Correlate metrics
- ✓ Automate investigation



DEVOPS TIP

Use CLI when dashboards are down.
The API never lies.



OBSERVABILITY GOLDEN RULE

Without logs, metrics are blind.
Without context, logs are noisy.



REMEMBER

Observe → Correlate

DEVSECOPS CLI TOOLKIT

SECURITY TOOLS A DEVOPS ENGINEER CAN OPERATE DIRECTLY FROM THE TERMINAL.



DEVSECOPS

1. TRIVY

Comprehensive vulnerability scanner for containers, filesystems, repos, and configurations.

Image scanning

```
trivy image nginx:latest
```

Filesystem scanning

```
trivy fs ./project
```

Repository scanning

```
trivy repo https://github.com/org/repo
```

Configuration scanning

```
trivy config ./k8s/
```

2. SYFT

Generate and inspect Software Bill of Materials (SBOMs).

Generate SBOM for an image

```
syft nginx:latest -o table
```

Generate SBOM for filesystem

```
syft dir:. -o json > sbom.json
```

Inspect packages

```
syft packages sbom.json
```

Output formats

```
-o table | -o json | -o spdx-json
```

3. GRYPE

Scan SBOMs and container images for vulnerabilities.

Scan image

```
grype nginx:latest
```

Scan SBOM file

```
grype sbom:sbom.json
```

Scan directory (via syft)

```
syft dir:. -o json | grype -
```

List only high/critical

```
grype nginx:latest -q "severity >= high"
```

4. GITLEAKS

Detect and prevent secrets exposed in Git repositories.

Scan current directory

```
gitLeaks detect
```

Scan a Git repository

```
gitLeaks detect --source .
```

Custom config

```
gitLeaks detect --config gitLeaks.toml
```

Baseline (ignore known findings)

```
gitLeaks detect --baseline .gitLeaks.baseline
```

5. OPENSSSL

TLS, certificates, and encryption toolkit.

Check certificate details

```
openssl x509 -in cert.pem -noout -text
```

Check certificate expiry

```
openssl x509 -in cert.pem -noout -enddate
```

Test TLS connection

```
openssl s_client -connect example.com:443
```

Generate private key

```
openssl genrsa -out key.pem 2048
```

Generate CSR

```
openssl req -new -key key.pem -out req.csr
```

Verify certificate chain

```
openssl verify -CAfile ca.crt cert.pem
```

Convert certificate formats

```
openssl x509 -in cert.pem -out cert.crt
```

Check TLS protocol and cipher

```
openssl s_client -connect example.com:443 -tls1_2
```

Use cases

- ✓ TLS debugging
- ✓ Certificate validation
- ✓ Certificate expiry checks
- ✓ Secure connections testing

6. SECURITY UTILITIES

sha256sum

Verify file integrity

```
sha256sum file.txt
```

Verify multiple files

```
sha256sum -c checksums.txt
```

ssh-keygen

Manage SSH keys

```
ssh-keygen -t ed25519 -C "devops@veriqta.com"
```

List public key

```
cat ~/.ssh/id_ed25519.pub
```

gpg

Encrypt and sign data

```
gpg --full-generate-key
```

Encrypt file

```
gpg -c secret.txt
```

Decrypt file

```
gpg secret.txt.gpg
```

Common uses

- ✓ File integrity verification
- ✓ Secure key management
- ✓ Data encryption and decryption
- ✓ Digital signing and verification

7. DEVSECOPS CLI USE CASES



CONTAINER SECURITY

Scan container images for vulnerabilities before deployment.



DEPENDENCY VULNERABILITIES
Identify vulnerable dependencies in projects and images.



SECRET DETECTION

Detect exposed secrets in repositories and prevent leaks.



SBOM WORKFLOWS

Generate SBOMs, scan them, and track software components.



TLS DEBUGGING

Inspect certificates, test TLS connections, and troubleshoot issues.



CI/CD SECURITY GATES

Integrate security scans into pipelines and block unsafe deployments.

8. TYPICAL DEVSECOPS CLI WORKFLOW



BUILD IMAGE

```
docker build -t myapp:latest .
```



SCAN IMAGE

```
trivy image myapp:latest
```



GENERATE SBOM

```
syft myapp:latest -o json > sbom.json
```



SCAN SBOM

```
grype sbom:sbom.json
```



SCAN REPO

```
gitLeaks detect
```



SECURE DEPLOY

Pass security gates and deploy safely.

★ Automate everything in CI/CD for continuous security.
Scan → Detect → Validate → Approve → Deploy.

9. BEST PRACTICES

- ✓ Scan early and scan often.
- ✓ Keep tools and databases updated.
- ✓ Fail pipelines on high/critical vulnerabilities.
- ✓ Do not commit secrets. Use secret stores.
- ✓ Verify TLS everywhere.
- ✓ Use least privilege access.
- ✓ Automate and document security checks.



DEVOPS TIP

Security is not a one-time task. It is continuous, automated, and enforced.



SECURITY REMINDER

A secure pipeline delivers a secure platform.



REMEMBER

Visibility → Detection → Action → Protection
That is DevSecOps.

JSON, YAML, API, AND DATA CLI TOOLKIT

DEVOPS ENGINEERS CONSTANTLY TRANSFORM
MACHINE-READABLE OUTPUT.



JSON



YAML



API

jq 1. JQ - JSON PROCESSING

Powerful JSON processor for the command line.

Feature	Example
.	<code>cat data.json jq .</code>
Object selection	<code>jq '.user.name'</code>
Array selection	<code>jq '.items[0]'</code>
Filters	<code>jq '.items[] select(.id == 1)'</code>
Pipes	<code>cat data.json jq '.items[]' jq .name</code>
select()	<code>jq 'select(.status=="Running")'</code>
map()	<code>jq '.items map(.name)'</code>
Raw output	<code>jq -r '.user.name'</code>

Example: Extract all pod names

```
kubectl get pods -o json | jq -r '.items[].metadata.name'
```

yq 2. YQ - YAML PROCESSING

YAML processor for reading, filtering and modifying YAML files.

Task	Example
Read YAML	<code>yq e '.app.name' config.yaml</code>
Filter YAML	<code>yq e '.spec.template.spec' deployment.yaml</code>
Modify YAML	<code>yq e '.replicas = 3' -i deployment.yaml</code>
Convert YAML to JSON	<code>yq e -o=json file.yaml</code>
Convert JSON to YAML	<code>yq e -P file.json</code>

Example: Get container image

```
yq e '.spec.template.spec.containers[0].image' deployment.yaml
```

curl 3. CURL - API REQUESTS

Command-line tool for making HTTP requests.

Method	Example
GET	<code>curl https://api.example.com/data</code>
POST	<code>curl -X POST https://api.example.com/data</code>
PUT	<code>curl -X PUT https://api.example.com/data/1</code>
PATCH	<code>curl -X PATCH https://api.example.com/data/1</code>
DELETE	<code>curl -X DELETE https://api.example.com/data/1</code>
Headers	<code>curl -H "Content-Type: application/json" ...</code>
Authentication	<code>curl -H "Authorization: Bearer TOKEN" ...</code>
JSON payloads	<code>curl -X POST -H "Content-Type: application/json" -d '{"name":"app"}' https://api.example.com/data</code>
Status codes	<code>curl -o /dev/null -w "%(http_code)" https://api.example.com/data</code>

4. POWERFUL WORKFLOWS WITH JQ



Pretty-print JSON

```
curl https://api.example.com/data | jq .
```

Format JSON output for readability.



Extract API Values

```
curl https://api.example.com/users | jq -r '.users[].name'
```

Extract specific fields from API responses.



Parse Kubernetes Output

```
kubectl get pods -o json | jq -r '.items[] | [.metadata.name, .status.phase] | @tsv'
```

Turn Kubernetes JSON into clean, usable data.



Parse AWS Output

```
aws ec2 describe-instances | jq -r '.Reservations[].Instances[] | .InstanceId'
```

Extract values from AWS CLI JSON output.



Transform Configuration

```
cat config.json | jq '.replicas = 3 | .debug = true'
```

Modify JSON/YAML config programmatically.

5. COMBINING TOOLS FOR REAL-WORLD AUTOMATION



COMBINE CURL + JQ

```
curl -s https://api.example.com/users | jq '.users[] | {id: .id, name: .name}'
```

Fetch API data and extract only the fields you need.



COMBINE KUBECTL + JQ

```
kubectl get pods -A -o json | jq -r '.items[] | [.metadata.namespace, .metadata.name, .status.phase] | @tsv'
```

Query Kubernetes cluster and transform output into clean tables.



COMBINE AWS + JQ

```
aws s3api list-buckets | jq -r '.Buckets[] | .Name'
```

Extract AWS resource information with precision.

6. AUTOMATION-FRIENDLY STRUCTURED OUTPUT

- ✓ Use JSON and YAML for all automation tasks.
- ✓ Prefer structured output over plain text.
- ✓ Combine CLI tools to build powerful pipelines.
- ✓ Validate and transform data before using it.
- ✓ Make scripts idempotent and reliable.



API / CLI



FETCH
(curl / aws / kubectl)



PROCESS
(jq / yq)



OUTPUT
(JSON / YAML / TSV)



AUTOMATE
(Scripts / Tools / Pipelines)



7. BEST PRACTICES

- ✓ Always use `-s` (silent) and `-f` (fail) with curl in scripts.
- ✓ Use `jq -r` for raw output in scripts and automation.
- ✓ Validate JSON/YAML before using in pipelines.
- ✓ Keep queries simple, test incrementally.
- ✓ Log and handle errors gracefully.



DEVOPS TIP

The best DevOps engineers speak fluent JSON, YAML, and API.



REMEMBER

Good data in → Good automation.
Bad data in → Broken everything.
Transfer